

UNIT-3: Linux Files

File concept:

A file can be defined as a container which is used to store information or it can also be termed as a sequence of characters.

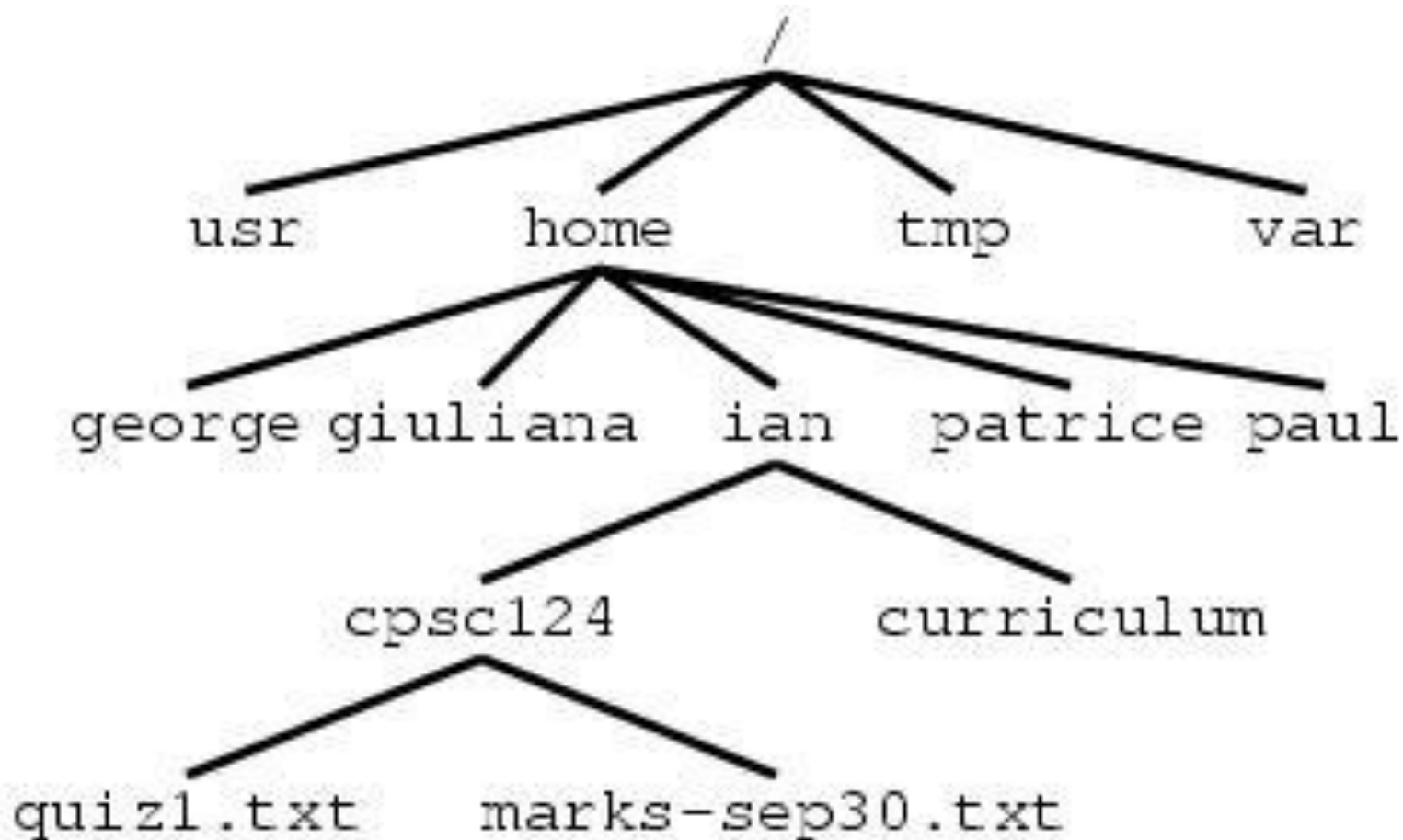
Kernel it self is treated as a file.

A file can be divided into 3 ways,

- 1) Files which are present on the operating system level and other is the user level.
- 2) At the operating system level, a file is stored in the form of a data structure of a disk slice or other storage device which allows to store the data (hard drive).
- 3) At the user level file system is nothing but, hierarchical collection of directories, sub-directories and files. This is known as the directory hierarchy

File system structure

The Unix file system is a hierarchical structure that allows users to store information by name. At the top of the hierarchy is the root directory, which always has the name /. A typical Unix file system might look like:



File system structure

- Files in UNIX or POSIX systems are stored in tree-like hierarchical file system.
- The root of a file system is the root (“/”) directory.
- The leaf nodes of a file system tree are either empty directory files or other types of files.
- **Absolute path name of a file consists of the names of all the directories, starting from the root.**
- **Ex: /usr/divya/a.out**
- **Relative path name may consist of the “.” and “..” characters. These are references to current and parent directories respectively.**
- **Ex: ../../.login denotes .login file which may be found 2 levels up from the current directory**
- A file name may not exceed NAME_MAX characters (14 bytes) and the total number of characters of a path name may not exceed PATH_MAX (1024 bytes).
- POSIX.1 defines _POSIX_NAME_MAX and _POSIX_PATH_MAX in <limits.h> header

File system structure

- File name can be any of the following character set only

A to Z a to z 0 to 9 _

- Path name of a file is called the **hardlink**.
- A file may be referenced by more than one path name if a user creates one or more hard links to the file using **ln command**.

ln /usr/foo/path1 /usr/prog/new/n1

- If the -s option is used, then it is a symbolic (soft) link .

The following files are commonly defined in most UNIX systems

FILE	Use
/etc	Stores system administrative files and programs
/etc/passwd	Stores all user information's
/etc/shadow	Stores user passwords
/etc/group	Stores all group information
/bin	Stores all the system programs like cat, rm, cp, etc.

File system structure

/dev	Stores all character device and block device files
/usr/include	Stores all standard header files.
/usr/lib	Stores standard libraries
/tmp	Stores temporary files created by program

Inode

Each file is associated with an *inode*, which is identified by an integer number, often referred to as an *i-number* or *inode number*.

Inodes store information about files and directories (folders), such as file

- 1.ownership
- 2 .type of file
- 3.Group owner
- 4.File access permissions
- 5.Date and time of last access
- 6.Number of links
- 7.Size of the file
- 8.Address of the blocks where file is physically present

A file's inode number can be found using the `ls -li` command. The `ls -l` command displays some of the inode contents for each file.

File Type

A file in a UNIX or POSIX system may be one of the following types:

- regular file
- directory file
- FIFO file
- Character device file
- Block device file

Regular file

- A regular file may be either a text file or a binary file
- These files may be read or written to by users with the appropriate access permission
- Regular files may be created, browsed through and modified by various means such as text editors or compilers, and they can be removed by specific system commands

File Types

Directory file

- It is like a folder that contains other files, including sub-directory files.
- It provides a means for users to organize their files into some hierarchical structure based on file relationship or uses.
- Ex: **/bin directory contains all system executable programs, such as cat, rm, sort**
- A directory may be created in UNIX by the **mkdir command**
Ex: **mkdir /usr/foo/xyz**
- A directory may be removed via the **rmdir command**
Ex: **rmdir /usr/foo/xyz**

The content of directory may be displayed by the **ls command**

File Types

Device file

Block device file

It represents a physical device that transmits data a block at a time.

Ex: hard disk drives and floppy disk drives

A physical device may have both block and character device files representing it for different access methods.

An application program in turn may choose to transfer data by either a character-based(via character device file) or block-based(via block device file)

- A device file is created in UNIX via the **mknod command**

Ex: **mknod** **/dev/cdsk** **c** **115** **5**

Character device file

It represents a physical device that transmits data in a character-based manner.

Ex: line printers, modems, and consoles

Here , c - character device file

115 - major device number

5 - minor device number

File Types

It is a special pipe device file which provides a temporary buffer for two or more processes to communicate by writing data to and reading data from the buffer.

- The size of the buffer is fixed to PIPE_BUF.
- Data in the buffer is accessed in a first-in-first-out manner.
- The buffer is allocated when the first process opens the FIFO file for read or write
- The buffer is discarded when all processes close their references (stream pointers) to the FIFO file.
- Data stored in a FIFO buffer is temporary.

- A FIFO file may be created via the **mkfifo command**.

The following command creates a FIFO file (if it does not exists)

mkfifo /usr/prog/fifo_pipe

The following command creates a FIFO file (if it does not exists)

mknod /usr/prog/fifo_pipe p

- FIFO files can be removed using **rm command**.

File Types

Symbolic link file

- BSD UNIX & SV4 defines a symbolic link file.
- A symbolic link file contains a path name which references another file in either local or remote file system.
- POSIX.1 does not support symbolic link file type
- A symbolic link may be created in UNIX via the **ln command**
- Ex: **ln -s /usr/divya/original /usr/raj/slink**
- It is possible to create a symbolic link to reference another symbolic link.
- **rm, mv and chmod commands will operate only on the symbolic link arguments directly and not on the files that they reference.**

File Types

The type of the file can be determined with the macros given in table

Macro name	file type
S_ISREG	regular file
S_ISDIR	directory file
S_ISCHAR	character special file
S_ISBLK	block special file
S_ISFIFO	fifo file
S_ISLNK	symbolic link
S_ISSOCK	socket file

The Standard I/O

The Standard I/O:

The functions which are already present in the library is known as the standard I/O functions.

Three file streams are automatically opened when a program is started. They are `stdin`, `stdout` and `stderr`.

These are declared in `stdio.h` and represent the standard input, output and error output, respectively, which

correspond to the low level file descriptors 0, 1 and 2.

- `fopen`, `fclose`
- `fread`, `fwrite`
- `fflush`
- `fseek`
- `fgetc`, `getc`, `getchar`
- `fputc`, `putc`, `putchar`
- `fgets`, `gets`

The Standard I/O

Opening a Stream: `fopen()`

```
#include <stdio.h>
```

```
FILE *fopen(const char *filename, const char *mode);
```

The `fopen` library function is the analog of the low level open system call.

`fopen` opens the file named by the `filename` parameter and associates a stream with it.

The mode parameter specifies how the file is to be opened. It's one of the following strings:

- "r" or "rb" Open for reading only
- "w" or "wb" Open for writing, truncate to zero length
- "a" or "ab" Open for writing, append to end of file
- "r+" or "rb+" or "r+b" Open for update (reading and writing)
- "w+" or "wb+" or "w+b" Open for update, truncate to zero length
- "a+" or "ab+" or "a+b" Open for update, append to end of file

The Standard I/O

The `b` indicates that the file is a binary file rather than a text file. Note that, unlike DOS, UNIX doesn't make a distinction between text and binary files. It treats all files exactly the same, effectively as binary files. It's also important to note that the mode parameter must be a string, and not a character. Always use `"r"`, and never `'r'`.

If successful, `fopen` returns a non-null `FILE *` pointer. If it fails, it returns the value `NULL`, defined in `stdio.h`.

The Standard I/O

Fread :

#include <stdio.h>

size_t fread(void *ptr, size_t size, size_t nitems, FILE *stream);

The fread library function is used to read data from a file stream. Data is read into a data buffer given by ptr from the stream, stream. Both fread and fwrite deal with data records. These are specified by a record size, size, and a count, nitems, of records to transfer. It returns the number of items (rather than the number of bytes) successfully read into the data buffer. At the end of a file, fewer than nitems may be returned, including zero.

The Standard I/O

fwrite

#include <stdio.h>

size_t fwrite (const void *ptr, size_t size, size_t nitems, FILE *stream);

The fwrite library call has a similar interface to fread. It takes data records from the specified data buffer and writes them to the output stream. It returns the number of records successfully written.

Important Note that **fread** and **fwrite** are not recommended for use with **structured data**.

The Standard I/O

fclose:

An open stream is closed by calling fclose.

```
#include <stdio.h>
```

```
int fclose(FILE *fp);
```

Returns: upon successful completion 0 is returned. otherwise, EOF is returned and the global variable errno is set to indicate the error.

Any buffered output data is flushed before the file is closed. Any input data that may be buffered is discarded. If the standard I/O library had automatically allocated a buffer for the stream, that buffer is released.

The Standard I/O

```
#include <stdio.h>
```

```
int fflush(FILE *stream);
```

The fflush library function causes all outstanding data on a file stream to be written immediately. You can use this to ensure that, for example, an interactive prompt has been sent to a terminal before any attempt to read a response. It's also useful for ensuring that important data has been committed to disk before continuing. You can sometimes use it when you're debugging a program to make sure that the program is writing data and not hanging. Note that an implied flush operation is carried out when fclose is called, so you don't need to call fflush before fclose.);

The Standard I/O

fseek

#include <stdio.h>

int fseek(FILE *stream, long int offset, int whence);

The `fseek` function is the file stream equivalent of the `lseek` system call. It sets the position in the stream for the next read or write on that stream. The meaning and values of the `offset` and `whence` parameters are the same as those we gave for `lseek` above. However, where `lseek` returns an `off_t`, `fseek` returns an integer: 0 if it succeeds, -1 if it fails, with `errno` set to indicate the error. So much for standardization!

fgetc, getc, getchar

```
#include <stdio.h>
```

```
int fgetc(FILE *stream);
```

```
int getc(FILE *stream);
```

```
int getchar();
```

The `fgetc` function returns the next byte, as a character, from a file stream.

When it reaches the end of the file or there is an error, it returns EOF. You must use `ferror` or `feof` to distinguish the two cases.

The `getc` function is equivalent to `fgetc`, except that you can implement it as a macro, in which case the stream argument must not have side effects (i.e. it can't affect variables that are neither local nor passed to the functions as parameters). Also, you can't then use the address of `getc` as a function pointer.

The `getchar` function is equivalent to `getc(stdin)` and reads the next character from the standard input.

fputc, putc, putchar

```
#include <stdio.h>
```

```
int fputc(int c, FILE *stream);
```

```
int putc(int c, FILE *stream);
```

```
int putchar(int c);
```

The fputc function writes a character to an output file stream. It returns the value it has written, or EOF on failure.

As with fgetc/getc, the function putc is equivalent to fputc, but you may implement it as a macro.

The putchar function is equivalent to putc(c,stdout), writing a single character to the standard output. Note that putchar takes and getchar returns characters as ints, not char. This allows the end of file (EOF) indicator to take the value -1 , outside the range of character numbers codes.

fgets, gets

```
#include <stdio.h>
```

```
char *fgets(char *s, int n, FILE *stream);
```

```
char *gets(char *s);
```

The fgets function reads a string from an input file stream. It writes characters to the string pointed to by s until a newline is encountered, n-1 characters have been transferred or the end of file is reached, whichever occurs first. Any newline encountered is transferred to the receiving string and a terminating null byte, \0, is added. Only a maximum of n-1 characters are transferred in any one call, because the null byte must be added to finish the string, and make up the n bytes.

Formatted I/O

Formatted Output:

Formatted output is handled by the four printf functions.

```
#include <stdio.h>
```

```
int printf(const char *restrict format, ...);
```

```
int fprintf(FILE *restrict fp, const char *restrict format, ...);
```

Both return: number of characters output if OK, negative value if output error.

```
int sprintf (char *restrict buf, const char *restrict format, ...);
```

```
int snprintf (char *restrict buf, size_t n, const char *restrict format, ...);
```

Both return: number of characters stored in array if OK, negative value if encoding error.

The printf function writes to the standard o/p, fprintf writes to the specified stream& sprintf places the formatted characters in the array buf. The sprintf function automatically appends a null byte at the end of the array, but this null byte is not included in the return value.

Formatted I/O

Formatted Input:

Formatted input is handled by the three scanf functions.

```
#include <stdio.h>
```

```
int scanf (const char *restrict format, ...);
```

```
int fscanf (FILE *restrict fp, const char *restrict format, ...);
```

```
int sscanf (const char *restrict buf, const char *restrict format, ...);
```

All three return: number of input items assigned,

EOF if input error or end of file before any conversion.

Library functions

List of functions

Function	Description
Clearerr	check and reset stream status
Fclose	close a stream
Fdopen	stream open functions
Feof	check and reset stream status
Ferror	check and reset stream status
Fflush	flush a stream
Fgetc	get next character or word from input stream
Fgetpos	reposition a stream
Fgets	get a line from a stream
Fileno	return the integer descriptor of the argument stream
Fopen	stream open functions
Fprintf	formatted output conversion
Fpurge	flush a stream
Fputc	output a character or word to a stream
Fputs	output a line to a stream

Other Stream Functions

Other library functions use either stream parameters or the standard streams **stdin**, **stdout**, **stderr**:

- **fgetpos** Get the current position in a file stream.
- **fsetpos** Set the current position in a file stream.
- **ftell** Return the current file offset in a stream.
- **rewind** Reset the file position in a stream.
- **freopen** Reuse a file stream.
- **setvbuf** Set the buffering scheme for a stream.
- **remove** Equivalent to **unlink**, unless the **path** parameter is a directory in which case it's equivalent to **rmdir**.

Stream Errors

To indicate an error, many of the **stdio** library functions return out of range values, such as null pointers or the constant EOF.

In these cases, the error is indicated in the external variable **errno**:

```
#include <errno.h>

extern int errno;
```

You can also interrogate the state of a file stream to determine whether an error has occurred, or the end of file has been reached.

```
#include <stdio.h>

int ferror(FILE *stream);
int feof(FILE *stream);
void clearerr(FILE *stream);
```

Stream Errors

The **ferror** function tests the error indicator for a stream and returns non-zero if it is set, zero otherwise.

The **feof** function tests the end-of-file indicator within a stream and returns non-zero if it is set zero otherwise.

You use it like this:

```
if (feof(some_stream))  
    /* We're at the end */
```

The **clearerr** function clears the end-of-file and error indicators for the stream to which stream points.

Streams and File Descriptors

Each file stream is associated with a low level file descriptor.

You can mix low-level input and output operations with higher level stream operations, but this is generally unwise.

The effects of buffering can be difficult to predict.

```
#include <stdio.h>

int fileno(FILE *stream);
FILE *fdopen(int fildes, const char *mode);
```


Kernel Support for files

In UNIX system V, the kernel maintains a file table that has an entry of all opened files and also there is an inode table that contains a copy of file inodes that are most recently accessed.

The steps involved are : 1. The kernel will search the process descriptor table and look for the first unused entry. If an entry is found, that entry will be designated to reference the file .The index of the entry will be returned to the process as the file descriptor of the opened file.

2. The kernel will scan the file table in its kernel space to find an unused entry that can be assigned to reference the file.

If an unused entry is found the following events will occur:

- The process file descriptor table entry will be set to point to this file table entry.
- The file table entry will be set to point to the inode table entry, where the inode record of the file is stored.

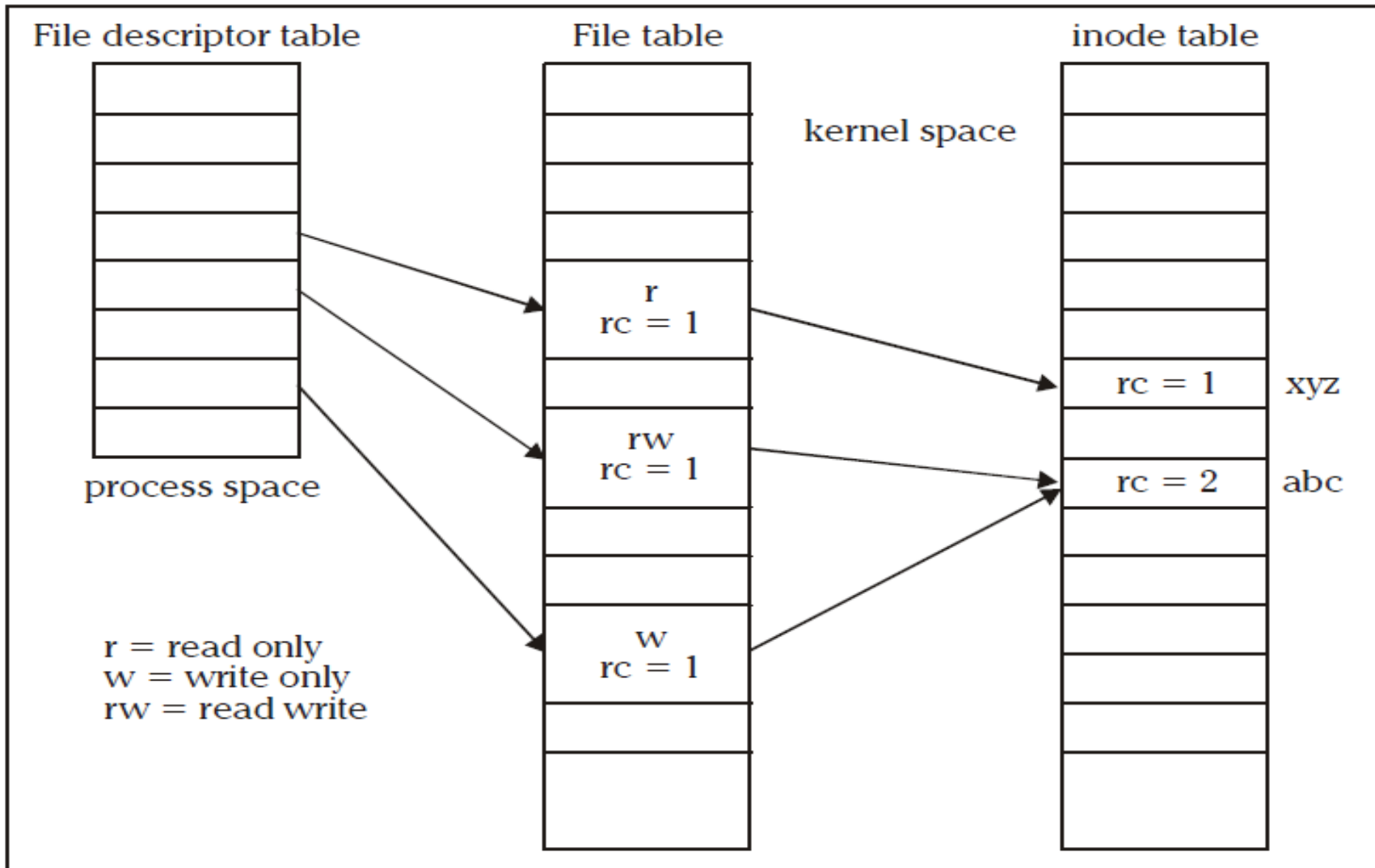
System calls

- The file table entry will contain the current file pointer of the open file. This is an offset from the beginning of the file where the next read or write will occur.
- The file table entry will contain an open mode that specifies that the file opened is for read only, write only or read and write etc. This should be specified in open function call.
- The reference count (rc) in the file table entry is set to 1. Reference count is used to keep track of how many file descriptors from any process are referring the entry.
- The reference count of the in-memory inode of the file is increased by 1. This count specifies how many file table entries are pointing to that inode.

If either (1) or (2) fails, the **open system call returns -1 (failure/error)**

System calls

Data Structure for File Manipulation



System calls

Normally the reference count in the file table entry is 1, if we wish to increase the rc in the file table entry, this can be done using fork, dup, dup2 system call. When a open system call is succeeded, its return value will be an integer (file descriptor). Whenever the process wants to read or write data from the file, it should use the file descriptor as one of its argument.

The following events will occur whenever a process calls the close function to close the files that are opened.

1. The kernel sets the corresponding file descriptor table entry to be unused.
2. It decrements the rc in the corresponding file table entry by 1, if rc not equal to 0 go to step 6.
3. The file table entry is marked as unused.
4. The rc in the corresponding file inode table entry is decremented by 1, if rc value not equal to 0 go to step 6.

System calls

5. If the hard link count of the inode is not zero, it returns to the caller with a success status otherwise it marks the inode table entry as unused and de-allocates all the physical disk storage of the file.
6. It returns to the process with a 0 (success) status.

File descriptors

In computer programming, a **file descriptor** (FD) is an abstract indicator for accessing a [file](#). The term is generally used in [POSIX operating systems](#).

In POSIX, a file descriptor is an integer, specifically of the [C](#) type *int*. There are three standard POSIX file descriptors, corresponding to the three [standard streams](#), which presumably every process (save perhaps a [daemon](#)) should expect to have:

Integer value	Name
0	Standard input (<i>stdin</i>)
1	Standard output (<i>stdout</i>)
2	Standard error (<i>stderr</i>)

Generally, a file descriptor is an index for an entry in a [kernel](#)-resident [array data structure](#) containing the details of open files. In POSIX this data structure is called a file descriptor table, and each [process](#) has its own file descriptor table. The process passes the file descriptor to the kernel through

File descriptors

a [system call](#), and the kernel will access the file on behalf of the process. The process itself cannot read or write the file descriptor table directly.

On [Linux](#), the set of file descriptors open in a process can be accessed under the path `/proc/PID/fd/`, where PID is the [process identifier](#).

In [Unix-like](#) systems, file descriptors can refer to any [Unix file type](#) named in a file system. As well as regular files, this includes [directories](#), [block](#) and [character devices](#) (also called "special files"), [Unix domain sockets](#), and [named pipes](#). File descriptors can also refer to other objects that do not normally exist in the file system, such as [anonymous pipes](#) and [network sockets](#).

Low –level file access

UNIX system calls are used to manage the file system, control processes, and to provide inter process communication.

Types of system calls in UNIX:

- | | |
|------------------|----------|
| 1.Open | 2.creat |
| 3.read | 4.write |
| 5.Close | 6.lseek |
| 7.stat | 8.fstat |
| 9.ioctl | 10.umask |
| 11 .dup and dup2 | 12.fcntl |

Low –level file access

open function:

This is used to establish a connection between a process and a file i.e. it is used to open an existing file for data transfer function or else it may be also be used to create a new file.

The prototype of open function is

```
#include<sys/types.h>
```

```
#include<sys/fcntl.h>
```

```
int open(const char *pathname, int accessmode, mode_t permission);
```

- If successful, open returns a nonnegative integer representing the open file descriptor.
- If unsuccessful, open returns -1.
- The first argument is the name of the file to be created or opened. This may be an absolute pathname or relative pathname.

Low –level file access

- If the given pathname is symbolic link, the open function will resolve the symbolic link reference to a non symbolic link file to which it refers.
- The second argument is access modes, which is an integer value that specifies how actually the file should be accessed by the calling process.
- Generally the access modes are specified in <fcntl.h>. Various access modes are:
 - **O_RDONLY** - open for reading file only
 - **O_WRONLY** - open for writing file only
 - **O_RDWR** - opens for reading and writing file.
- There are other access modes, which are termed as access modifier flags, and one or more of the following can be specified by bitwise-ORing them with one of the above access mode flags to alter the access mechanism of the file.

Low –level file access

O_APPEND - Append data to the end of file.

O_CREAT - Create the file if it doesn't exist

O_EXCL - Generate an error if O_CREAT is also specified and the file already exists.

O_TRUNC - If file exists discard the file content and set the file size to zero bytes.

O_NONBLOCK - Specify subsequent read or write on the file should be non-blocking.

O_NOCTTY - Specify not to use terminal device file as the calling process control terminal.

Low –level file access

creat Function

The prototype of creat is

```
#include <sys/types.h>
```

```
#include<unistd.h>
```

```
int creat(const char *pathname, mode_t mode);
```

- Returns: file descriptor opened for write-only if OK, -1 on error.
- The first argument pathname specifies name of the file to be created.
- The second argument mode_t, specifies permission of a file to be accessed by owner group and others.
- The creat function can be implemented using open function as:

```
#define creat(path_name, mode)
```

```
open (pathname, O_WRONLY | O_CREAT | O_TRUNC, mode);
```

Low –level file access

read

- The read function fetches a fixed size of block of data from a file referenced by a given file descriptor.

- The prototype of read function is:

```
#include<sys/types.h> #include<unistd.h> size_t read(int fdesc, void *buf,  
size_t nbyte);
```

- If successful, read returns the number of bytes actually read.
- If unsuccessful, read returns -1.
- The first argument is an integer, fdesc that refers to an opened file.
- The second argument, buf is the address of a buffer holding any data read.
- The third argument specifies how many bytes of data are to be read from the file.
- The size_t data type is defined in the <sys/types.h> header and should be the same as unsigned int.

Low –level file access

- There are several cases in which the number of bytes actually read is less than the amount requested:
- When reading from a regular file, if the end of file is reached before the requested number of bytes has been read. For example, if 30 bytes remain until the end of file and we try to read 100 bytes, read returns 30. The next time we call read, it will return 0 (end of file).
- When reading from a terminal device. Normally, up to one line is read at a time.
- When reading from a network. Buffering within the network may cause less than the requested amount to be returned.
- When reading from a pipe or FIFO. If the pipe contains fewer bytes than requested, read will return only what is available.

Low –level file access

write

- The write system call is used to write data into a file.
- The write function puts data to a file in the form of fixed block size referred by a given file descriptor.

- The prototype of write is

```
#include<sys/types.h>
```

```
#include<unistd.h>
```

```
ssize_t write(int fdesc, const void *buf, size_t size);
```

- If successful, write returns the number of bytes actually written.
- If unsuccessful, write returns -1.
- The first argument, fdesc is an integer that refers to an opened file.
- The second argument, buf is the address of a buffer that contains data to be written.
- The third argument, size specifies how many bytes of data are in the buf argument.
- The return value is usually equal to the number of bytes of data successfully written to a file. (*size value*)

Low –level file access

close

- The close system call is used to terminate the connection to a file from a process.
- The prototype of the close is

```
#include<unistd.h>      int close(int fdesc);
```
- If successful, close returns 0. If unsuccessful, close returns -1.
- The argument fdesc refers to an opened file.
- Close function frees the unused file descriptors so that they can be reused to reference other files. This is important because a process may open up to OPEN_MAX files at any time and the close function allows a process to reuse file descriptors to access more than OPEN_MAX files in the course of its execution.
- The close function de-allocates system resources like file table entry and memory buffer allocated to hold the read/write.

Low –level file access

fcntl

- The fcntl function helps a user to query or set flags and the close-on-exec flag of any file descriptor.

- The prototype of fcntl is

```
#include <fcntl.h>
```

```
int fcntl(int fdesc, int cmd, ...);
```

- The first argument is the file descriptor.
- The second argument cmd specifies what operation has to be performed.
- The third argument is dependent on the actual cmd value.
- The possible cmd values are defined in <fcntl.h> header.

Low –level file access

cmd value	Use
F_GETFL	Returns the access control flags of a file descriptor fdesc
F_SETFL	Sets or clears access control flags that are specified in the third argument to fcntl. The allowed access control flags are O_APPEND & O_NONBLOCK
F_GETFD	Returns the close-on-exec flag of a file referenced by fdesc. If a return value is zero, the flag is off; otherwise on.
F_SETFD	Sets or clears the close-on-exec flag of a fdesc. The third argument to fcntl is an integer value, which is 0 to clear the flag, or 1 to set the flag
F_DUPFD	Duplicates file descriptor fdesc with another file descriptor. The third argument to fcntl is an integer value which specifies that the duplicated file descriptor must be greater than or equal to that value. The return value of fcntl is the duplicated file descriptor

Low –level file access

lseek

- The lseek function is also used to change the file offset to a different value.
- Thus lseek allows a process to perform random access of data on any opened file.

- The prototype of lseek is

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
off_t lseek(int fdesc, off_t pos, int whence);
```

- On success it returns new file offset, and -1 on error.
- The first argument fdesc, is an integer file descriptor that refer to an opened file.
- The second argument pos, specifies a byte offset to be added to a reference location in deriving the new file offset value.
- The third argument whence, is the reference location.

Low –level file access

Whence value	Reference location
SEEK_CUR	Current file pointer address
SEEK_SET	The beginning of a file
SEEK_END	The end of a file

Low –level file access

stat, fstat

- The stat and fstat function retrieves the file attributes of a given file.
- The only difference between stat and fstat is that the first argument of a stat is a file pathname, where as the first argument of fstat is file descriptor.
- The prototypes of these functions are

```
#include<sys/stat.h>
```

```
#include<unistd.h>
```

```
int stat(const char *pathname, struct stat *statv);
```

```
int fstat(const int fd, struct stat *statv);
```

The second argument to stat and fstat is the address of a struct stat-typed variable which is defined in the <sys/stat.h> header.

Low –level file access

- Its declaration is as follows:

```
struct stat
```

```
{
```

```
dev_t st_dev; /* file system ID */
```

```
ino_t st_ino; /* file inode number */
```

```
mode_t st_mode; /* contains file type and permission */
```

```
nlink_t st_nlink; /* hard link count */
```

```
uid_t st_uid; /* file user ID */
```

```
gid_t st_gid; /* file group ID */
```

```
dev_t st_rdev; /*contains major and minor device#*/
```

```
off_t st_size; /* file size in bytes */
```

```
time_t st_atime; /* last access time */
```

```
time_t st_mtime; /* last modification time */
```

```
time_t st_ctime; /* last status change time */ };
```

Low –level file access

The return value of both functions is

- 0 if they succeed
- -1 if they fail
- *errno contains an error status code*

Low –level file access

ioctl

#include <unistd.h>

int ioctl(int fildes, int cmd, ...);

ioctl is a bit of a rag–bag of things. It provides an interface for controlling the behavior of devices, their

descriptors and configuring underlying services. Terminals, file descriptors, sockets, even tape drives may

have ioctl calls defined for them and you need to refer to the specific device's man page for details. POSIX

only defines ioctl for streams, which are beyond the scope of this book.

ioctl performs the function indicated by cmd on the object referenced by the descriptor fildes. It may take an

optional third argument depending on the functions supported by a particular device.

File and Record Locking

Linux system allow multiple processes to read and write the same file concurrently , but it results data in a file can be overridden by another process. To avoid this drawback linux supports file and record locking.

File locking is applicable only for regular files. It allows a process to impose a lock on a file so that other process can not modify the file until it is unlocked by the process.

Record locking is used to describe the ability of a process to prevent other processes from modifying a region of a file while the first process is reading or modifying that portion of the file. It is **byte-range locking**, since it is a range of a file (possibly the entire file) that is locked.

File and Record Locking

fcntl Record Locking:

```
#include <fcntl.h>
```

```
int fcntl(int filedes, int cmd, ... /* struct flock *flockptr */ );
```

Returns: depends on cmd if OK, -1 on error.

Here cmd is F_GETLK, F_SETLK, or F_SETLKW.

Flockptr is a pointer to an flock structure.

```
struct flock {  
    short l_type;  
    off_t l_start;  
    short l_whence;  
    off_t l_len;  
    pid_t l_pid;  
};
```

File and Record Locking

The type of lock desired:

F_RDLCK (a shared read lock),

F_WRLCK (an exclusive write lock), or F_UNLCK (unlocking a region).

The starting byte offset of the region being locked or unlocked (l_start and l_whence).

The size of the region in bytes (l_len).

The ID (l_pid) of the process holding the lock that can block the current process (returned by F_GETLK only).

Rules about the specification of the region to be locked or unlocked:

Locks can start and extend beyond the current end of file, but cannot start or extend before the beginning of the file.

If l_len is 0, it means that the lock extends to the largest possible offset of the file. This allows us to lock a region starting anywhere in the file, up through and including any data that is appended to the file

File and Record Locking

To lock the entire file, we set `l_start` and `l_whence` to point to the beginning of the file and specify a length (`l_len`) of 0.

Types of locks:

1. Shared lock:

read locks are shared locks. Any no. of process can have read lock on the same object at a time.

2. Exclusive lock :

Write locks are exclusive locks. When one process puts write lock on the object, no other read lock or write lock is allowed on the same object.

File and Record Locking

Commands for the fcntl function:

F_GETLK: Determine whether the lock described by flockptr is blocked by some other lock. It does only verification and for not applying locks.

F_SETLK: Set the lock described by flockptr.

F_SETLKW: This command is a blocking version of F_SETLK. If the requested read lock or write lock cannot be granted because another process currently has some part of the requested region locked, the calling process is put to sleep.

Function to lock or unlock a region of a file:

```
#include <fcntl.h>
```

```
int lock_reg (int fd, int cmd, int type, off_t offset, int whence, off_t len)
```

```
{ struct flock lock;
```

```
lock.l_type = type;
```

```
lock.l_start = offset;
```

File and Record Locking

```
lock.l_whence = whence;  
lock.l_len = len;  
return(fcntl( fd, cmd, &lock));  
}
```

Implied Inheritance and Release of Locks:

Locks are associated with a process and a file.

when a process terminates, all its locks are released. whenever a descriptor is closed, any locks on the file referenced by that descriptor for that process are released.

Locks are never inherited by the child across a fork.

Place a write lock on an entire file:

```
#include <unistd.h>  
#include <fcntl.h>  
int lockfile (int fd) {
```

File and Record Locking

```
struct flock fl;  
    fl.l_type = F_WRLCK;  
    fl.l_start = 0;  
    fl.l_whence = SEEK_SET;  
    fl.l_len = 0;  
    return(fcntl(fd, F_SETLK, &fl));  
}
```

Advisory locking:

In Unix, advisory locking is done with `flock()` and `lockf()` commands.

Both *fcntl* and *flock* offer *advisory* locking. Advisory locking is locking that requires the cooperation of participating processes.

advisory locking is sometimes called *unenforced* locking.

File and Record Locking

Mandatory locking:

Is the kernel enforced file locking, as opposed to the more usual cooperative file locking used to guarantee sequential access to files among processes.

A file is marked as a candidate for mandatory locking by setting the group-id bit in its file mode but removing the group-execute bit.

Mandatory locks can only be applied via the `fcntl()/lockf()` locking interface.

If a process has locked a region of a file with a mandatory read lock, then other processes are permitted to read from that region. If any of these processes attempts to write to the region it will block until the lock is released.

If a process has locked a region of a file with a mandatory write lock, all attempts to read or write to that region block until the lock is released.

File and Record Locking

Advisory versus Mandatory Locking

Mandatory locking only protects those portions of a file that are locked. Other portions of the file that are not locked may be accessed according to normal UNIX system file permissions.

Advisory locking is more efficient because a record lock check does not have to be performed for every I/O request.

File and Directory Management

The standard libraries and system calls provide complete control over the creation and maintenance of files

and directories.

chmod, fchmod

- The chmod and fchmod functions change file access permissions for owner, group & others as well as the set_UID, set_GID and sticky flags.
- A process must have the effective UID of either the super-user/owner of the file.

- The prototypes of these functions are

```
#include<sys/types.h> #include<sys/stat.h> #include<unistd.h> int
```

```
chmod(const char *pathname, mode_t flag); int fchmod(int fdesc, mode_t flag);
```

- The pathname argument of chmod is the path name of a file whereas the fdesc argument of fchmod is the file descriptor of a file.
- The chmod function operates on the specified file, whereas the fchmod function operates on a file that has already been opened.
- To change the permission bits of a file, the effective user ID of the process must be equal to the owner ID of the file, or the process must have super-user permissions. The mode is specified as the bitwise OR of the constants shown below.

File and Directory Management

chown

A superuser can change the owner of a file using the chown system call.

```
#include <unistd.h>
```

```
int chown(const char *path, uid_t owner, gid_t group);
```

The call uses the numeric values of the user and group IDs (culled from getuid and getgid calls) and a constant

which can restrict who can change file ownership. The owner and group of a file are changed if the

appropriate privileges are set.

File and Directory Management

unlink, link, symlink

We can remove a file using unlink.

#include <unistd.h>

int unlink(const char *path);

int link(const char *path1, const char *path2);

int symlink(const char *path1, const char *path2);

The unlink system call removes the directory entry for a file decrements the link count for it. It returns 0 if the unlinking was successful, -1 on an error. You must have write and execute permissions in the directory where the file has its directory entry for this call to function.

File and Directory Management

mkdir, rmdir

We can create and remove directories using the mkdir and rmdir system calls.

#include <sys/stat.h>

int mkdir(const char *path, mode_t mode);

The mkdir system call is used for creating directories and is the equivalent of the mkdir program. mkdir

makes a new directory with path as its name. The directory permissions are passed in the parameter mode and

are given as in the O_CREAT option of the open system call and, again, subject to umask.

#include <unistd.h>

int rmdir(const char *path);

The rmdir system call removes directories, but only if they are empty. The rmdir program uses this system call to do its job.

File and Directory Management

chdir, getcwd

A program can navigate directories in much the same way as a user moves around the UNIX file system. As

we use the `cd` command in the shell to change directory, so a program can use the `chdir` system call.

```
#include <unistd.h>
```

```
int chdir(const char *path);
```

A program can determine its current working directory by calling the `getcwd` function.

```
#include <unistd.h>
```

```
char *getcwd(char *buf, size_t size);
```

The `getcwd` function writes the name of the current directory into the given buffer, `buf`. It returns null if the

directory name would exceed the size of the buffer (an `ERANGE` error), given

as the parameter size. It returns buf on success.

Scanning Directories

The directory functions are declared in a header file, `dirent.h`. They use a structure, `DIR`, as a basis for directory manipulation. A pointer to this structure, called a **directory stream (a `DIR *`)**, acts in much the same way as a file stream (`FILE *`) does for regular file manipulation. Directory entries themselves are returned in `dirent` structures, also declared in `dirent.h`, as one should never alter the fields in the `DIR` structure directly.

We'll review these functions:

- `opendir`, `closedir`
- `readdir`
- `telldir`
- `seekdir`

Scanning Directories

opendir

The `opendir` function opens a directory and establishes a directory stream. If successful, it returns a pointer to a `DIR` structure to be used for reading directory entries.

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
DIR *opendir(const char *name);
```

`opendir` returns a null pointer on failure. Note that a directory stream uses a low-level file descriptor to access the directory itself, so `opendir` could fail with too many open files.

readdir

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
struct dirent *readdir(DIR *dirp);
```

Scanning Directories

The `readdir` function returns a pointer to a structure detailing the next directory entry in the directory stream `dirp`. Successive calls to `readdir` return further directory entries. On error, and at the end of the directory, `readdir` returns `NULL`. POSIX compliant systems leave `errno` unchanged when returning `NULL` at end of directory and set it when an error occurs.

Note that `readdir` scanning isn't guaranteed to list all the files (and subdirectories) in a directory if there are other processes creating and deleting files in the directory at the same time.

Scanning Directories

telldir

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
long int telldir(DIR *dirp);
```

The `telldir` function returns a value that records the current position in a directory stream. You can use this in subsequent calls to `seekdir` to reset a directory scan to the current position.

seekdir

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
void seekdir(DIR *dirp, long int loc);
```

The `seekdir` function sets the directory entry pointer in the directory stream given by `dirp`. The value of `loc`, used to set the position, should have been obtained from a prior call to `telldir`.

Scanning Directories

closedir

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
int closedir(DIR *dirp);
```

The `closedir` function closes a directory stream and frees up the resources associated with it. It returns 0 on success and `-1` if there is an error.

In the next program, `printdir.c`, we put together a lot of the file manipulation functions to create a simple directory listing. Each file in a directory is listed on a line by itself. Each subdirectory has its name, followed by a slash and the files in it are listed indented by four spaces.